

Final Evolution Lab — Complete Deployment Guide

This guide covers all deployment steps for Final Evolution Lab, including Unreal Engine setup, Pixel Streaming infrastructure, web frontend, testing, and iOS deployment.

Table of Contents

- 1. [Prerequisites](#)
- 2. [Phase 1: Unreal Engine Setup](#)
- 3. [Phase 2: Streaming Infrastructure](#)
- 4. [Phase 3: Web App Deployment](#)
- 5. [Phase 4: Testing](#)
- 6. [Phase 5: iOS Deployment](#)
- 7. [Cloud Deployment](#)
- 8. [Troubleshooting](#)

Prerequisites

Tool	Version	Required For
Unreal Engine	5.4+	Game server
Node.js	20+	Signalling & Frontend
Docker + Compose	Latest	Container deployment
Python 3.10+		AI asset pipeline
Xcode 16+		iOS builds
GPU (NVIDIA)		UE5 rendering

Phase 1: Unreal Engine Setup

1.1 Install Unreal Engine 5.x

On Linux (recommended for server)

```
# Option A: Epic Games Launcher (via Lutris/Wine)
# Download from: https://www.unrealengine.com/download

# Option B: Build from source (recommended for Linux servers)
git clone https://github.com/EpicGames/UnrealEngine.git
cd UnrealEngine
git checkout 5.4 # or latest 5.x tag
./Setup.sh
./GenerateProjectFiles.sh
make
```

On macOS (for development)

1. Download Epic Games Launcher from <https://www.unrealengine.com/download>
2. Install UE 5.4+ through the launcher
3. The engine will be at: `/Users/Shared/Epic Games/UE_5.4/`

On Windows

1. Download Epic Games Launcher
2. Install UE 5.4+ through the Library tab
3. Default path: `C:\Program Files\Epic Games\UE_5.4\`

1.2 Open the Project

```
# Navigate to the project
cd /path/to/rork-final-evolution-lab/UnrealStarter/BasketballGame

# Open in UE Editor (Linux)
/path/to/UnrealEngine/Engine/Binaries/Linux/UnrealEditor \
  "$(pwd)/FinalEvolutionLab.uproject"

# Open in UE Editor (macOS)
open -a "/Users/Shared/Epic Games/UE_5.4/Engine/Binaries/Mac/UnrealEditor.app" \
  "$(pwd)/FinalEvolutionLab.uproject"
```

1.3 Import AI-Generated Assets

The project has 49 AI-generated assets (23 animations, 12 environments, 9 props, 5 characters) that need importing.

Automatic Import (Editor Python)

1. Open the project in UE Editor
2. Enable the **Python Editor Script Plugin** (Edit → Plugins → search “Python”)
3. Run the import script:

Edit → Execute Python Script → Select:

`UnrealStarter/BasketballGame/EditorPython/fel_import_ai_assets.py`

4. Or via command line:

bash

`UnrealEditor FinalEvolutionLab.uproject \`

```
-ExecutePythonScript="EditorPython/fel_import_ai_assets.py" \
-unattended -nopause -nosplash
```

Manual Import (if Python script fails)

1. In Content Browser, right-click → Import
2. Navigate to GeneratedAssets/ folder
3. Import files to these content paths:
 - deepmotion/*.fbx → /Game/FEL/Generated/deepmotion/
 - meshy/*.glb → /Game/FEL/Generated/meshy/
 - luma/*.fbx → /Game/FEL/Generated/luma/
4. For FBX animations: set “Import as Skeletal” and “Import Animations”

1.4 Cook for Linux Server

```
# From UE installation directory
UE_ROOT="/path/to/UnrealEngine"
PROJECT="/path/to/UnrealStarter/BasketballGame/FinalEvolutionLab.uproject"

# Cook content for Linux
"$UE_ROOT/Engine/Binaries/Linux/UnrealEditor" "$PROJECT" \
  -run=cook \
  -targetplatform=Linux \
  -server \
  -map=AllMaps \
  -iterate \
  -unattended

# Package for Linux Shipping server
"$UE_ROOT/Engine/Build/BatchFiles/RunUAT.sh" BuildCookRun \
  -project="$PROJECT" \
  -noP4 \
  -platform=Linux \
  -clientconfig=Shipping \
  -serverconfig=Shipping \
  -cook -allmaps -build -stage -pak -archive \
  -archivedirectory="$(pwd)/PackagedBuild" \
  -PixelStreamingEnabled
```

After cooking, copy the packaged build to streaming/docker/build/ .

Phase 2: Streaming Infrastructure

2.1 Quick Start (Native)

```
# Start all services
./scripts/start_services.sh

# Services will be available at:
# Frontend:      http://localhost:3000
# Signalling:    ws://localhost:8888
# Health:        http://localhost:8888/healthz
# Modes API:     http://localhost:8888/api/modes
```

2.2 Docker Deployment

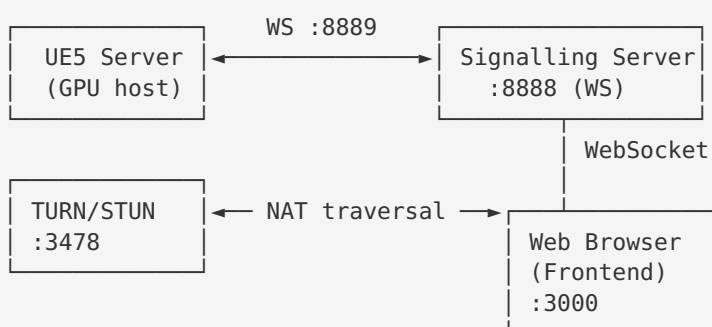
```
cd streaming

# Start signalling + TURN + frontend (no UE server yet)
docker compose up --build -d signalling coturn frontend

# Check status
docker compose ps
docker compose logs -f signalling

# Full stack (after UE build is in streaming/docker/build/)
docker compose up --build -d
```

2.3 Service Architecture



2.4 Configuration

Edit `streaming/config/streaming_config.json` for production:

```
{
  "signallingUrl": "wss://your-domain.com:8888",
  "turnServer": "turn:your-domain.com:3478",
  "turnUser": "fel",
  "turnPass": "<secure-password>",
  "maxPlayers": 10,
  "resolution": { "width": 1920, "height": 1080 },
  "targetFps": 60
}
```

Phase 3: Web App Deployment

3.1 Development

```
cd streaming/frontend
npm install
npm run dev # Starts Vite dev server with HMR
```

3.2 Production Build

```
cd streaming/frontend
npm install
npm run build    # Output in dist/
npm run preview  # Test production build locally
```

3.3 Deploy to Hosting

Netlify

```
# From project root (netlify.toml already configured)
npx netlify deploy --prod --dir=streaming/frontend/dist
```

Vercel

```
cd streaming/frontend
npx vercel --prod
```

Nginx (self-hosted)

```
server {
    listen 80;
    server_name fel.yourdomain.com;
    root /var/www/fel/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /ws {
        proxy_pass http://localhost:8888;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    location /api/ {
        proxy_pass http://localhost:8888;
    }
}
```

3.4 Environment Variables

Create `.env` in `streaming/frontend/` :

```
VITE_SIGNALLING_URL=wss://your-domain.com:8888
```

Phase 4: Testing

4.1 Automated Tests

```
# Run full deployment test suite
./scripts/test_deployment.sh

# Run game mode verification
./scripts/test_game_modes.sh

# Install WebSocket test dependency (optional)
pip install websockets
```

4.2 Manual Testing Checklist

For each of the 16+1 game modes:

#	Mode	Category	Verify
1	Street · 1v1	Basketball	Spawn, scoring, 1v1 AI
2	Dunk Contest	Basketball	Dunk meter, trick scoring
3	Street · 3v3	Basketball	Team spawn, 3v3 mechanics
4	Karate · 1v1	Combat	Strike/block, health bars
5	Karate · Endless	Combat	Wave spawning, survival
6	Baseball · Ballpark	Field	Batting, pitching, fielding
7	Football · Kick Return	Field	Return mechanics, tackles
8	Soccer · Stadium	Field	Ball physics, goals
9	Golf · Links	Precision	Swing meter, course holes
10	Tennis · Court	Court	Serve, rally, scoring
11	Volleyball · Sand	Court	Bump/set/spike
12	Gymnastics · Floor	Performance	Routine scoring, dismounts
13	Brain Brawl	Academy	Quiz mechanics, timers
14	Surf · Line	Board	Wave riding, tricks
15	Skate · Park	Board	Trick combos, score
16	Snow · Line	Board	Downhill, trick scoring
17	Sovereign Shop	Shop	Browse, purchase flow

4.3 Exercise System Testing

1. Open Exercise Panel from main menu
 2. For each of the 23 exercises:
 - Verify 3D animation plays correctly
 - Check correct muscle group highlighting
 - Confirm rep counter works
 - Test PRQ score integration
-

Phase 5: iOS Deployment

5.1 Prerequisites

- **macOS** with Xcode 16+ installed
- **Apple Developer Account** (\$99/year) enrolled in the Apple Developer Program
- **iPhone** running iOS 17+ for testing
- **Certificates & Provisioning Profiles** configured in Xcode

5.2 Xcode Project Setup

```
# Navigate to iOS project
cd /path/to/rork-final-evolution-lab/ios

# Install CocoaPods dependencies (if applicable)
pod install

# Open workspace
open FinalEvolutionLab.xcworkspace
# Or if no workspace:
open FinalEvolutionLab.xcodeproj
```

5.3 Configure Signing

1. Open Xcode → Select project in navigator
2. Select the **FinalEvolutionLab** target
3. Go to **Signing & Capabilities** tab
4. Set **Team** to your Apple Developer team
5. Set **Bundle Identifier**: `com.yourcompany.finalevolutionlab`
6. Ensure **Automatically manage signing** is checked

5.4 Configure Streaming URL

Update the streaming server URL in the iOS app:

```
// In ios/FinalEvolutionLab/Config/StreamingConfig.swift (or equivalent)
struct StreamingConfig {
    static let signallingURL = "wss://your-domain.com:8888"
    static let turnServer = "turn:your-domain.com:3478"
    static let turnUsername = "fel"
    static let turnCredential = "fel_streaming_2026"
}
```


5.5 Build & Run on Device

1. Connect iPhone via USB
2. Select your device in the scheme toolbar
3. Press **⌘+R** to build and run
4. First run will require trusting the developer profile on the device:
 - Settings → General → VPN & Device Management → Trust

5.6 TestFlight Deployment

Step 1: Archive

1. Select **Any iOS Device** as the build target
2. **Product** → **Archive** (⌘+Shift+B won't work, must use Archive)
3. Wait for the build to complete

Step 2: Upload to App Store Connect

1. In the **Organizer** (Window → Organizer), select the new archive
2. Click **Distribute App**
3. Select **App Store Connect** → **Upload**
4. Follow the wizard (keep defaults for most options)
5. Wait for processing (~15-30 minutes)

Step 3: Configure in App Store Connect

1. Go to <https://appstoreconnect.apple.com>
2. Navigate to **My Apps** → **Final Evolution Lab**
3. Click **TestFlight** tab
4. The uploaded build should appear under **iOS Builds**
5. Click the build → **Manage Compliance** → Select encryption status
6. Add test information (contact email, notes)

Step 4: Invite Testers

1. **Internal Testing** (up to 100 team members):
 - TestFlight → Internal Testing → Create Group
 - Add team members by Apple ID
2. **External Testing** (up to 10,000 testers):
 - TestFlight → External Testing → Create Group
 - Add testers by email or share public link
 - Requires Beta App Review (~24-48 hours)

Step 5: Tester Experience

1. Testers receive email invitation
2. Install TestFlight app from App Store
3. Open invitation link or redeem code
4. Install and test the app
5. Provide feedback through TestFlight

5.7 UE5 iOS Build (Alternative — Native UE Rendering)

If you want a native UE5 iOS build instead of streaming:

```
# On macOS with UE5 installed
UE_ROOT="/Users/Shared/Epic Games/UE_5.4"
PROJECT="/path/to/UnrealStarter/BasketballGame/FinalEvolutionLab.uproject"

# Cook for iOS
"$UE_ROOT/Engine/Build/BatchFiles/RunUAT.sh" BuildCookRun \
  -project="$PROJECT" \
  -platform=IOS \
  -clientconfig=Shipping \
  -cook -allmaps -build -stage -pak -archive \
  -archivedirectory="$(pwd)/IOSBuild"
```

Then open the generated Xcode project in `IOSBuild/` and follow standard Xcode deployment.

Cloud Deployment

AWS (g4dn instances)

```
# 1. Build and push Docker images
aws ecr create-repository --repository-name fel-streaming
docker compose build
docker tag streaming-signalling:latest <account>.dkr.ecr.<region>.amazonaws.com/fel-
signalling:latest
docker push <account>.dkr.ecr.<region>.amazonaws.com/fel-signalling:latest

# 2. Launch GPU instance
aws ec2 run-instances \
  --instance-type g4dn.xlarge \
  --image-id ami-0abcdef1234567890 \
  --key-name your-key \
  --security-groups fel-streaming-sg

# 3. SSH in and run
ssh -i your-key.pem ubuntu@<ip>
docker compose up -d
```

GCP (T4 GPU)

```
gcloud compute instances create fel-streaming \
  --machine-type=n1-standard-4 \
  --accelerator=type=nvidia-tesla-t4,count=1 \
  --image-family=ubuntu-2204-lts \
  --boot-disk-size=100GB
```

Troubleshooting

Signalling server won't start

```
# Check if port is in use
lsof -i :8888
# Kill and restart
./scripts/stop_services.sh
./scripts/start_services.sh
```

Frontend build fails

```
cd streaming/frontend
rm -rf node_modules dist
npm install
npm run build
```

WebSocket connection refused

- Verify signalling server is running: `curl http://localhost:8888/healthz`
- Check firewall rules allow ports 8888, 8889, 3478
- For remote connections, ensure TURN server is configured

UE5 streamer won't connect

- Verify signalling server is listening on port 8889
- Check UE5 launch args include `-PixelStreamingIP=<host> -PixelStreamingPort=8889`
- Look at UE5 log for Pixel Streaming errors

Docker GPU errors

```
# Install NVIDIA Container Toolkit
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -
o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctl runtime configure --runtime=docker
sudo systemctl restart docker
```

iOS build signing errors

- Verify Apple Developer account is active
- Check Xcode → Preferences → Accounts → Manage Certificates
- Clean build folder: Product → Clean Build Folder (⌘+Shift+K)
- Delete derived data: `rm -rf ~/Library/Developer/Xcode/DerivedData`